

NH-VOICE · 01

Vyngo Dispatch

Voice-Based Vehicle Search — Take-Home Submission

A buyer speaks a need in English/Hinglish. We transcribe, extract slots, hard-filter a real catalog, and speak back the top 3 — grounded strictly in catalog data, with zero invented facts.

Live demo: `python server.py` → `http://localhost:8000` (Chrome)

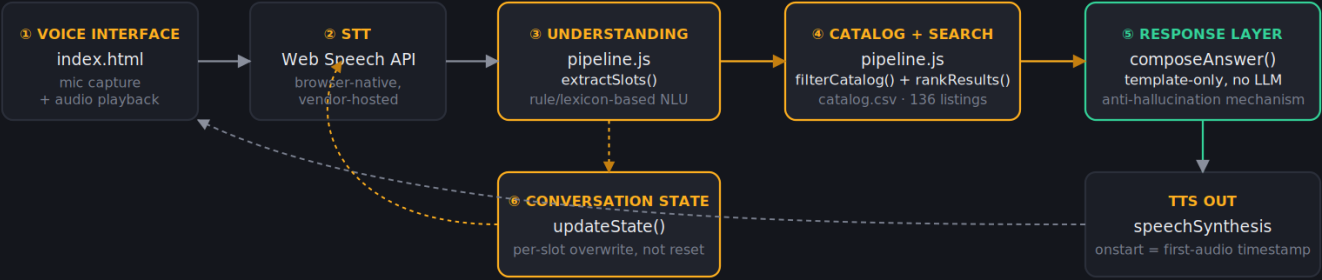
Eval: `node eval/run_eval.js` → 16/16 pass, 0 constraint violations

Repo layout: `pipeline.js` (shared core) · `index.html` (voice UI) · `eval/` (harness + results)

Seven components, one shared pipeline

Vyngo Voice Vehicle Search — Architecture

One process. Seven components. pipeline.js is shared verbatim by the live UI and the eval harness.



state persists turn-to-turn: corrections overwrite one slot,
"uska" follow-ups resolve against last-shown records

audio played back to buyer

7 EVAL HARNESS + LATENCY LOG — eval/run_eval.js (Node)

Imports the SAME pipeline.js the browser calls — require("../pipeline.js") — not a mock or reimplementation.
16 test utterances (eval/eval_utterances.json) → extractSlots() → filterCatalog() → rankResults() → composeAnswer()
Checks: (a) extracted slots match expected, (b) zero constraint violations across ALL filtered results,
(c) follow-ups return grounded answers, not the empty-context fallback. Prints pass rate + per-stage latency (ms).

16 / 16 PASS

0 constraint violations
eval/results.txt

same pipeline.js module, imported by both

Legend

- Core pipeline.js function (shared by UI + eval)
- Browser-only I/O boundary (STT/TTS/mic)
- Anti-hallucination boundary — response is template-built from catalog fields only

What each box hides / what would replace it — see README "Tradeoffs" and "Q&A prep" sections.
First component to replace in production: 2 STT (single function-boundary swap, see README scale answer).

The mechanism, not a prompt

Response layer builds the spoken sentence with template literals reading fields directly off catalog record objects (r.price, r.km_driven, r.year...). There is no free-text generation step between "matched record" and "spoken sentence" — so there's no channel for an invented number to enter.

What it catches

- Every price/km/year/payload spoken
- ...came from catalog.csv, verifiably
- Verified live by eval's constraint sweep

What it does NOT catch

- STT mishearing "5 lakh" as "50 lakh"
- — wrong filter applied, not a lie
- Mitigation: slots always shown on screen

TRADEOFF 1 OF 3

Rule-based NLU vs. LLM function-calling

DECISION

Regex/lexicon-based slot extraction (`extractSlots()`).

REJECTED ALTERNATIVE

LLM function-calling to extract budget/body-type/fuel/city/purpose.

WHY

Task is closed-vocabulary (5 known slot types) — LLM adds network latency + \$ per turn for no real gain here.

An LLM asked to extract a number can invent one — reintroduces the exact hallucination risk this assignment tests for.

Cost: brittle outside the lexicon ("compact hauler" won't map to `mini_truck` without a code change).

Cascaded STT -> NLU -> TTS vs. speech-to-speech

DECISION

Separate STT / understanding / TTS stages, each independently inspectable.

REJECTED ALTERNATIVE

An end-to-end speech-to-speech model (lower latency, better code-switching).

WHY

Speech-to-speech collapses the "slot state must be inspectable at every turn" requirement — no transcript/slot checkpoint to debug, correct, or eval against.

For a search task, correctness (never showing a 6L truck for a 5L budget) matters more than conversational naturalness.

TRADEOFF 3 OF 3

Web Speech API vs. a hosted vendor / open-source Whisper

DECISION

Browser-native Web Speech API for both STT and TTS.

REJECTED ALTERNATIVE

Hosted Whisper endpoint, or self-hosted Whisper-small.

WHY

Zero cost, zero setup, no server-side audio pipeline — fit the 12-16 hour budget.

Real cost: Chrome-only, needs network to Google's recognition service, Hindi/Hinglish accuracy noticeably worse than English.

Would flip this at scale (see next slide) once volume justifies a vendor SLA and GPU cost.

THE SCALE QUESTION

What breaks first at 100k conversations/month

1 Web Speech API's implicit rate limits

Free, undocumented-SLA service. Fix: paid STT vendor with a real SLA.

2 Single-process, in-memory conversation state

conversationState lives in one tab/process. Fix: keyed session store (Redis).

3 CSV-as-database

Linear scan is fine at 136 rows, not at real catalog scale + concurrent updates. Fix: indexed Postgres or a search engine.

4 STT/TTS vendor concurrency caps

Most vendors cap concurrent streams, not just monthly volume — bursts break this before compute does.

Not "add more servers" — pipeline.js is sub-millisecond per call (see eval log).
Bottlenecks are all at the I/O edges: STT/TTS vendor, session state, catalog store.